

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

PROJECT REPORT

A Comparative Study of Classical Planning and Reinforcement Learning for Maze Navigation

Trần Lê Hạ Đan

dan.tlh235483@sis.hust.edu.vn

Program: Data Science & Artificial Intelligence

Supervisor: Professor Đàm Quang Tuấn

School: School of Information and Communications Technology

HANOI, 07/2026

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION.....	1
1.1 Problem Statement.....	1
1.2 Background and Research Context	2
1.3 Objectives	2
1.4 Contributions	2
CHAPTER 2. LITERATURE REVIEW	3
2.1 Scope of Research	3
2.2 Related Work on Graph Search	3
2.3 Reinforcement Learning for Navigation	4
2.4 Reactive and Heuristic Agents.....	4
2.5 Research Gap and Motivation	4
CHAPTER 3. METHODOLOGY	5
3.1 Overview	5
3.2 Maze Generation	5
3.3 Classical Search Algorithms	6
3.4 Q-Learning Agents	6
3.4.1 Static weighted navigation	6
3.4.2 Dynamic navigation with local sensing	6
3.5 Reactive Baseline	7
3.6 Experimental Analysis	7
3.7 Evaluation Metrics.....	7
CHAPTER 4. THEORETICAL ANALYSIS	8
4.1 Complexity of Classical Search.....	8
4.2 Optimality and Suboptimality	8

4.3 Planning Versus Learning: Amortized Cost.....	9
4.4 Dynamic Environments as Non-Stationary MDPs.....	9
CHAPTER 5. NUMERICAL RESULTS.....	10
5.1 Evaluation Parameters.....	10
5.2 Simulation Method	10
5.3 Scalability of Classical Planning	10
5.3.1 Scaling with maze size	10
5.3.2 Scaling with wall density	11
5.4 Learning Stability and Convergence	11
5.4.1 Static weighted mazes	11
5.4.2 Dynamic mazes	12
5.5 Optimality Gap in Static Environments.....	12
5.6 Dynamic Environments: Planning Versus Learning	13
5.7 Synthesis: Answering the Research Question	15
CHAPTER 6. CONCLUSIONS	16
6.1 Summary of Findings.....	16
6.2 Limitations.....	16
6.3 Future Work.....	16
REFERENCE	17

ABSTRACT

Autonomous navigation in grid-based environments raises a central design question: should an agent compute a plan from a complete map, or should it learn a policy through interaction? Classical search methods such as A* (A-star), Breadth-first search (BFS), Depth-first search (DFS), and Dijkstra's algorithm provide reliable solutions on known maps, while reinforcement learning can adapt when the environment changes.

This report compares these two approaches on shared maze benchmarks. The experiments examine classical planning efficiency, learning stability, path optimality in weighted mazes, and dynamic-task performance with moving obstacles and bonus rewards. The results show that A* is consistently the most efficient classical planner, while Q-learning can match near-optimal path cost on small static mazes but becomes less reliable as maze size increases. In dynamic settings, A* replanning remains the most dependable method for reaching the goal, whereas Q-learning collects more bonuses at the cost of longer trajectories.

The main conclusion is that planning is preferable when the environment is static and fully known, while learning becomes attractive when rewards encourage exploration and repeated replanning is less practical. The report provides an empirical comparison of both paradigms under consistent conditions and highlights the trade-off between reliability and adaptability.

Source code available at: github.com/tlhdan/Pathfinding-Algorithms

LIST OF ABBREVIATIONS

Abbreviation	Definition
A*	A-star graph search algorithm with heuristic guidance
BFS	Breadth-first search
DFS	Depth-first search
MDP	Markov decision process
RL	Reinforcement learning

CHAPTER 1. INTRODUCTION

Pathfinding is a core problem in artificial intelligence, robotics, and game AI. Given a map, a start position, and a goal, an agent must decide how to move so as to reach the goal efficiently while avoiding obstacles. Two broad families of solutions exist. *Planning* methods construct or update a path from an explicit representation of the environment; they are grounded in graph theory and often provide formal guarantees. *Learning* methods acquire a policy through trial and error, storing experience in a value function rather than recomputing a plan from scratch at every decision.

The tension between these paradigms is not merely theoretical. In a warehouse robot operating on a fixed floor plan, a single A* query may suffice. In a game world where walls shift and rewards appear unpredictably, recomputing a global plan at every step may be too slow, yet a naively learned policy may fail before it has seen enough data. The central question for this report is therefore: *When is planning better than learning, and when does learning become necessary?*

This chapter introduces the problem, reviews the main approaches at a high level, and states the study objectives.

1.1 Problem Statement

Consider an agent navigating a two-dimensional grid maze. Cells may be free or blocked. In the *static* setting, the maze layout does not change after planning begins. In the *dynamic* setting, additional elements—moving obstacles and collectible bonuses—alter the effective cost landscape as the agent acts. The agent must reach a designated goal while minimizing movement cost, avoiding collisions, and optionally collecting bonuses.

Classical search algorithms assume that the environment is fully known and stationary. They excel at finding optimal or shortest paths but pay a computational price proportional to the number of states explored. Reinforcement learning treats navigation as a Markov decision process: the agent selects actions, receives rewards, and updates a Q-function that estimates long-term return. This paradigm can handle changing conditions without explicit replanning, but training requires many episodes and stability depends on hyperparameters and state representation.

The study therefore compares the conditions under which each paradigm is preferable. Rather than advocating a single best algorithm, the report evaluates the trade-off along measurable axes: runtime, search effort, path optimality, convergence reliability, and task success under dynamics.

1.2 Background and Research Context

Graph search has been studied extensively since Dijkstra’s algorithm [1] and the introduction of heuristic search in A* [2]. These methods are well understood: BFS finds shortest paths in unweighted graphs, Dijkstra handles non-negative edge costs, and A* reduces expansions via an admissible heuristic. Their limitations in dynamic worlds motivated replanning approaches such as D* Lite [3], which repair plans when costs change.

Reinforcement learning [4] offers a complementary view. Q-learning [5] learns action values without a model of transition dynamics. Tabular Q-learning on grid worlds remains a transparent baseline for studying convergence and optimality gaps. Existing work often evaluates planners and learners in isolation. Planning papers emphasize optimality and complexity; learning papers emphasize sample efficiency and generalization. This report addresses that gap by applying both families of methods to identical mazes and comparing them with shared metrics.

1.3 Objectives

The study pursues four objectives, each mapped to an experimental block.

The first objective is to quantify the *scalability* of classical planners. Runtime, nodes expanded, and maximum frontier size are measured as functions of maze size and wall density. The second objective is to assess *learning stability*. The third objective is to measure the *optimality gap* in static weighted mazes. The fourth objective is to compare *dynamic navigation strategies*: tabular Q-learning, A* replanning, and a reactive rule-based agent.

1.4 Contributions

The report makes the following contributions:

1. A reproducible experimental pipeline comparing four classical search algorithms and Q-learning on procedurally generated mazes with controlled size, density, and start–goal separation.
2. An empirical characterization of Q-learning convergence in static weighted mazes, showing near-optimal path costs when training succeeds but lower success rates as maze size increases.
3. A dynamic-environment benchmark that contrasts learned adaptation with A* replanning and a reactive baseline.
4. A synthesis of the results into practical guidance for choosing between planning and learning in maze navigation tasks.

CHAPTER 2. LITERATURE REVIEW

Chapter 1 established that the choice between planning and learning depends on environmental stability, computational constraints, and task structure. This chapter surveys the literature on graph search, reinforcement learning for navigation, and dynamic replanning, highlighting strengths and weaknesses that motivate the comparative experiments in later chapters.

2.1 Scope of Research

The scope of this report is deliberately constrained to two-dimensional grid mazes with four-connected movement. Classical algorithms include BFS, DFS, Dijkstra’s algorithm, and A*. The learning approach is tabular Q-learning with ϵ -greedy exploration, implemented in both a static weighted setting and a dynamic setting with local observations. Dynamic baselines include full A* replanning each step and a hand-crafted reactive policy.

The study does not address continuous state spaces, deep neural policies, multi-agent coordination, or partially observable mazes in the POMDP sense [7]. These exclusions keep the comparison interpretable while still capturing the essential planning–learning tension.

2.2 Related Work on Graph Search

Breadth-first search explores the frontier layer by layer and is optimal on unweighted graphs in terms of hop count [8]. Depth-first search uses less memory in some implementations but may produce highly suboptimal paths because it commits to deep branches before exploring alternatives. Dijkstra’s algorithm [1] generalizes BFS to non-negative edge weights and is optimal when edge costs reflect traversal expense. A* [2] adds a heuristic estimate of remaining cost; with an admissible heuristic such as Manhattan distance on grids, A* is optimal and typically expands far fewer nodes than uninformed search.

Comparative studies in robotics often treat A* and Dijkstra as interchangeable on uniform-cost grids; this report instead uses weighted mazes to separate hop-optimal paths (BFS) from cost-optimal paths (Dijkstra, A*). DFS serves as a cautionary baseline illustrating that completeness does not imply efficiency or optimality.

Dynamic replanning extends classical search to changing graphs. D* and D* Lite [3] repair paths incrementally when edge costs change, avoiding full recomputation. The A* replanning agent implemented in this repository is a simpler but instructive baseline: it replans from the current position at every timestep using an updated cost map that encodes obstacle and bonus locations. This approach is computationally heavy yet provides a strong reliability benchmark against which learned policies

can be judged.

2.3 Reinforcement Learning for Navigation

Markov decision processes formalize sequential decision-making under uncertainty [4]. Q-learning [5] learns action-value functions without requiring a model of the environment. Under standard conditions—finite state and action spaces, sufficient exploration, and appropriate learning rates—tabular Q-learning converges to the optimal Q-function.

Grid-world navigation is a canonical RL testbed because optimal policies can be verified by search. Prior work has combined learning with planning, for example through Dyna architectures or adaptive heuristic weights [9]. Such hybrids acknowledge that pure learning is sample-inefficient while pure planning is brittle under model error.

The dynamic Q-learning agent in this report uses a structured state comprising the agent position and local 5×5 patches of the maze, bonus map, and obstacle map at the current and previous timestep. This design follows the intuition that local perception is sufficient for reactive avoidance while position disambiguates corridors that look alike locally. The trade-off is state-space growth: larger mazes require more episodes to visit informative states.

2.4 Reactive and Heuristic Agents

Between full replanning and learned policies lie lightweight reactive rules. Bug algorithms, potential fields, and greedy heuristics execute in constant time per step but lack global optimality guarantees. The reactive agent evaluated here scores each legal move by Manhattan distance to the goal, penalizes cells occupied by predicted obstacle positions, and mildly prefers bonus cells. Such agents are easy to implement and require no training, making them attractive for embedded systems; the experiments test whether this simplicity translates into reliable performance when obstacles move unpredictably.

2.5 Research Gap and Motivation

Although textbooks [4], [8] treat search and RL as separate topics, practitioners face both simultaneously when building agents for changing environments. The gap addressed by this report is empirical: a side-by-side evaluation on shared mazes with common metrics, progressing from static scalability through learning stability and optimality to dynamic task performance.

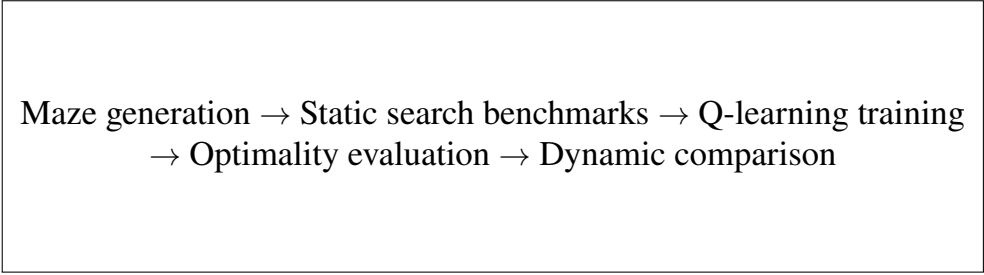
Prior comparisons often favor one paradigm by tuning unfairly—for example, evaluating RL after thousands of episodes against a planner given zero setup time. Here, training cost is explicitly measured via convergence episodes, and dynamic agents are compared on success rate and task rewards after fixed training budgets.

CHAPTER 3. METHODOLOGY

This chapter describes the experimental framework used to compare planning and learning on maze navigation tasks. The goal is to provide sufficient detail for reproduction without embedding source code in the narrative.

3.1 Overview

The experimental pipeline consists of five stages, illustrated conceptually in Figure 3.1. First, procedurally generated mazes are created with controlled size, wall density, and minimum start–goal separation. Second, classical search algorithms compute paths on static mazes while recording runtime and search statistics. Third, Q-learning agents are trained on static weighted mazes and evaluated for convergence and path cost. Fourth, path optimality metrics compare all methods against A* on identical instances. Fifth, dynamic scenarios introduce moving obstacles and bonuses; Q-learning, A* replanning, and the reactive agent are compared on task-oriented metrics.



```
graph LR; A[Maze generation] --> B[Static search benchmarks]; B --> C[Q-learning training]; C --> D[Optimality evaluation]; D --> E[Dynamic comparison];
```

Figure 3.1: High-level experimental pipeline. Each stage produces summary statistics and figures for the report.

All experiments use Python 3 with NumPy for grid operations. Random seeds are fixed per maze and training run to ensure reproducibility.

3.2 Maze Generation

Mazes are generated by recursive backtracking (randomized DFS carving) on a cell grid, followed by stochastic wall removal controlled by a *density* parameter in $(0, 1]$. Higher density retains more walls; lower density opens additional passages. Start and goal cells are sampled from free cells with Manhattan separation at least $\lfloor size/2 \rfloor$, ensuring non-trivial path length.

For optimality experiments, cell traversal costs are drawn uniformly from $[1, 50]$ on free cells, producing weighted grids where hop count and accumulated cost diverge. Dynamic experiments use unweighted free cells (cost 1) but introduce moving obstacles and bonus items.

Moving obstacles follow a fixed direction and reverse when blocked by a wall; they may swap positions with the agent under a defined collision rule. Bonuses are

removed upon collection and grant a positive reward during Q-learning training.

3.3 Classical Search Algorithms

Four search algorithms are implemented with a unified interface returning runtime, nodes expanded, and maximum priority-queue (frontier) size.

BFS uses a FIFO queue and is optimal for unweighted shortest-path hop count. DFS uses a LIFO stack; it is complete on finite graphs but not cost-optimal. Dijkstra’s algorithm uses a min-priority queue keyed by g -cost and handles non-negative weights. A* uses $f(n) = g(n) + h(n)$ with Manhattan distance as an admissible and consistent heuristic on grids.

In dynamic experiments, A* replanning invokes the static A* routine at every agent step on a *cost map* derived from the base maze: obstacle cells receive elevated cost, bonus cells receive reduced cost, encouraging detours that collect rewards while remaining traversable.

3.4 Q-Learning Agents

3.4.1 Static weighted navigation

The static agent maintains a tabular Q-function over state–action pairs (s, a) , where s is the agent’s grid coordinate and a is one of four movement actions. Rewards combine negative traversal cost, a penalty for invalid moves into walls, and a terminal bonus upon reaching the goal.

Training uses ϵ -greedy exploration with initial $\epsilon = 1$, decay 0.995 per episode, and minimum $\epsilon = 0.01$. Learning rate $\alpha = 0.1$ and discount $\gamma = 0.99$ are held fixed. Each training run lasts up to 1000 episodes (2000 for convergence logging). Every 50 episodes, the greedy policy is evaluated for successful goal reaching and path cost.

3.4.2 Dynamic navigation with local sensing

The dynamic agent extends the action space with a *wait* action. The state aggregates the agent position and four 5×5 binary patches: local maze layout, bonus indicators, current obstacle positions, and previous obstacle positions. This representation allows the agent to infer obstacle motion without a global map update at training time.

Rewards include a per-step penalty, collision penalties, bonus collection reward, goal reward, and penalties for invalid moves and waiting. Training and evaluation follow the same hyperparameters as the static case unless otherwise noted. After training, the greedy policy is rolled out in the dynamic simulator with moving obstacles.

3.5 Reactive Baseline

The reactive agent does not maintain a value function or global plan. At each step it enumerates legal moves (including wait), predicts next obstacle positions, and selects the move minimizing

$$\text{score} = d_{\text{Manhattan}}(\text{next}, \text{goal}) + 100 \cdot \mathbb{1}[\text{collision risk}] - 5 \cdot \mathbb{1}[\text{bonus at next cell}].$$

This rule encodes a greedy pursuit of the goal with hard avoidance of imminent collisions and mild bonus seeking. It requires no training episodes and serves as a lower-complexity alternative to both replanning and learning.

3.6 Experimental Analysis

Scalability of classical planning. Maze sizes $\{21, 41, 61, 81\}$ are tested at fixed density 0.75 with 100 random instances each. A complementary density sweep uses size 61 and densities $\{0.25, 0.5, 0.75, 1.0\}$. Metrics include runtime, nodes expanded, and max frontier size per algorithm.

Learning stability and convergence. Static and dynamic Q-learning are trained on sizes $\{11, 21, 31\}$ with 20 mazes and 5 independent training seeds each. Convergence is defined as three consecutive evaluation checkpoints with success and stable evaluation metric (path cost in static runs; cumulative reward in dynamic runs).

Optimality gap in static environments. On 20 weighted mazes per size, path cost and hop count from BFS, DFS, Dijkstra, A*, and trained Q-learning are compared. Cost ratio and length ratio are normalized by the A* optimum on each instance.

Dynamic environment comparison. For each size in $\{11, 21, 31\}$, 20 mazes $\times$ 5 runs per algorithm measure success rate, average steps, obstacle collisions, and bonuses collected for Q-learning, A* replanning, and the reactive agent. Dynamic Q-learning receives 1000 training episodes per instance before evaluation; A* and reactive agents require no training.

3.7 Evaluation Metrics

Table 3.1 summarizes the metrics used across blocks. Statistical summaries aggregate over maze instances and, for Q-learning, over training seeds.

Table 3.1: Evaluation metrics by experimental block.

Block	Primary metrics	Baseline
Scalability	Runtime, nodes expanded, frontier size	—
Convergence	Episodes to stabilize, convergence rate	—
Optimality	Cost ratio, length ratio, success rate	A* / Dijkstra
Dynamic	Success, steps, collisions, bonuses	A* replanning

CHAPTER 4. THEORETICAL ANALYSIS

This chapter complements the experimental description with a short theoretical view of the methods used. The focus is on why the empirical trade-offs in the following chapter should appear: computational complexity of search methods, optimality conditions, and convergence properties of tabular Q-learning.

4.1 Complexity of Classical Search

Let V denote the number of reachable free cells and E the number of adjacency edges in a grid maze. BFS and DFS each visit each reachable cell at most once, yielding time $\mathcal{O}(V)$ and space $\mathcal{O}(V)$ for explicit graph representations. With a binary heap, Dijkstra’s algorithm runs in $\mathcal{O}(E \log V)$; on a four-connected grid, $E \leq 4V$, so complexity is near-linear in maze area for uniform costs.

A* expands nodes in order of increasing $f = g + h$. With an admissible heuristic, A* is optimal. With a consistent heuristic—true of Manhattan distance on grids without diagonal moves—each cell is expanded at most once, and the number of expansions is bounded by the set of nodes with $f \leq f^*$, where f^* is the optimal solution cost. In practice, this subset is much smaller than V in structured mazes, which explains the reduced node counts relative to uninformed BFS and Dijkstra.

DFS exhibits different space behavior: the recursion or stack depth may reach $\mathcal{O}(V)$ in narrow corridors, and the maximum frontier size can grow large because the algorithm dives deep before backtracking. These properties predict higher frontier sizes for DFS than for best-first methods, particularly as maze size increases.

Wall density affects the effective branching factor. Sparse mazes (low density) contain many cycles, increasing the number of equal- f nodes and potentially reducing the heuristic advantage of A*. Perfect mazes (density 1.0) are trees with unique paths, simplifying search but lengthening shortest paths.

4.2 Optimality and Suboptimality

On weighted grids with non-negative costs, Dijkstra and A* return minimum-cost paths. BFS minimizes hop count and is cost-optimal only when all traversable cells share equal cost. DFS guarantees neither minimum hops nor minimum cost; it may wander through high-cost regions before reaching the goal.

Define the *cost ratio* of algorithm A on instance I as

$$\rho_A(I) = \frac{c_A(I)}{c^*(I)},$$

where $c^*(I)$ is the optimal cost returned by A* and $c_A(I)$ is the cost of the path produced by A . Similarly, the *length ratio* compares hop counts. Theory predicts $\rho_{A^*} = \rho_{\text{Dijkstra}} = 1$, $\rho_{\text{BFS}} \geq 1$ with equality only under uniform costs, and $\rho_{\text{DFS}} \gg 1$

in adversarial weight assignments.

For Q-learning in the static setting, the Bellman optimality equation is

$$Q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s', a')].$$

Tabular Q-learning converges to Q^* under suitable exploration and learning-rate conditions. The implemented agent uses a fixed α and ϵ -decay schedule; in practice, this often yields near-optimal greedy policies on small mazes, but convergence becomes less reliable as the state space grows.

4.3 Planning Versus Learning: Amortized Cost

Let C_{plan} be the cost of one full planning call and C_{act} the cost of selecting an action from a learned Q-table. For a path of length L , total planning cost is $\mathcal{O}(L \cdot C_{\text{plan}})$ if replanning each step, versus $\mathcal{O}(C_{\text{train}} + L \cdot C_{\text{act}})$ for learning, where C_{train} is the cumulative cost of training episodes.

In static environments with a single query, planning dominates whenever $C_{\text{plan}} \ll C_{\text{train}}$, which is usually true for tabular Q-learning requiring many episodes. In dynamic environments, repeated replanning can become expensive, while a learned policy can amortize training effort over many decisions.

4.4 Dynamic Environments as Non-Stationary MDPs

Moving obstacles render the transition dynamics non-stationary from the agent's perspective if it conditions only on current position. The dynamic Q-learning state includes local obstacle patches at two timesteps, approximating a Markov state for a stationary surrogate process. A* replanning sidesteps non-stationarity by re-optimizing against the current obstacle configuration each step, while the reactive agent relies on one-step lookahead and therefore lacks global reasoning.

CHAPTER 5. NUMERICAL RESULTS

This chapter reports the empirical results from the four experimental blocks and interprets them in light of the central comparison between planning and learning.

5.1 Evaluation Parameters

All experiments use procedurally generated mazes as described in Chapter 3. Unless stated otherwise, density is 0.75 for scalability tests, 0.8 for Q-learning and dynamic tests, and start–goal separation is at least $\lfloor \text{size}/2 \rfloor$ Manhattan steps. Classical scalability aggregates 100 mazes per configuration; optimality and dynamic tests use 20 mazes with five random seeds where stochastic training applies.

Statistical summaries report mean and standard deviation across instances. Q-learning convergence aggregates 100 training runs per maze size (20 mazes $\times$ 5 seeds). Dynamic comparison aggregates 100 runs per algorithm and size (20 mazes $\times$ 5 runs).

5.2 Simulation Method

Each experimental block follows a fixed protocol. Block 1 executes one search per algorithm per maze with instrumentation enabled. Block 2 logs evaluation checkpoints every 50 training episodes and marks convergence when three consecutive evaluations succeed with stable metrics. Block 3 computes optimal cost via A* and ratios for all successful paths. Block 4 trains dynamic Q-learning for 1000 episodes, then evaluates the three dynamic agents on the same maze–obstacle–bonus configurations with fixed random seeds.

5.3 Scalability of Classical Planning

5.3.1 Scaling with maze size

A* is consistently the fastest algorithm, with mean runtime remaining below 0.015 s even at size 81. BFS, DFS, and Dijkstra exhibit steeper growth: at size 81, mean runtimes are approximately 0.23 s, 0.27 s, and 0.24 s respectively. The log-scale presentation highlights the cost increase for uninformed methods as the number of reachable cells rises.

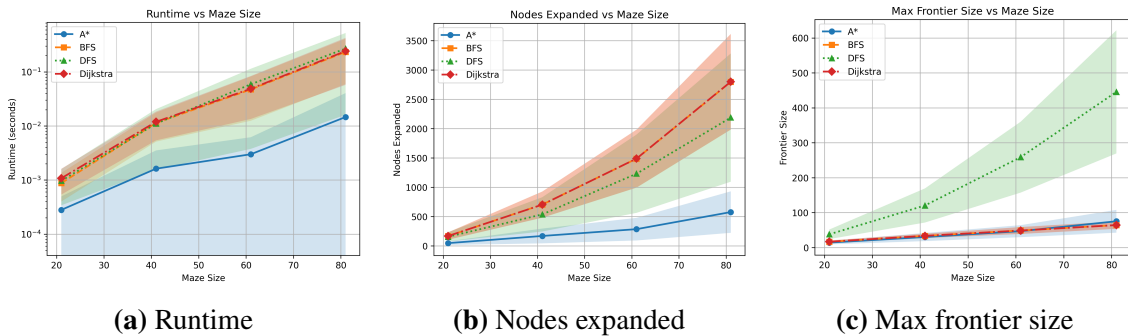


Figure 5.1: Scalability with maze size.

Table 5.1 summarizes the main values at the largest tested size. The results confirm that, for static planning on grids, informed search is not a minor optimization but a decisive factor at scale.

Table 5.1: Scalability at maze size 81 (density 0.75, mean over 100 instances).

Algorithm	Runtime (s)	Nodes expanded	Max frontier
A*	0.015	577	75
BFS	0.235	2799	64
Dijkstra	0.242	2801	64
DFS	0.270	2189	446

5.3.2 Scaling with wall density

The density sweep shows that A* maintains the lowest mean runtime at lower densities, while all algorithms converge toward similar timings in dense mazes where the state space collapses to a near-unique path. The same pattern appears in nodes expanded: A* remains the most efficient method across the tested densities.

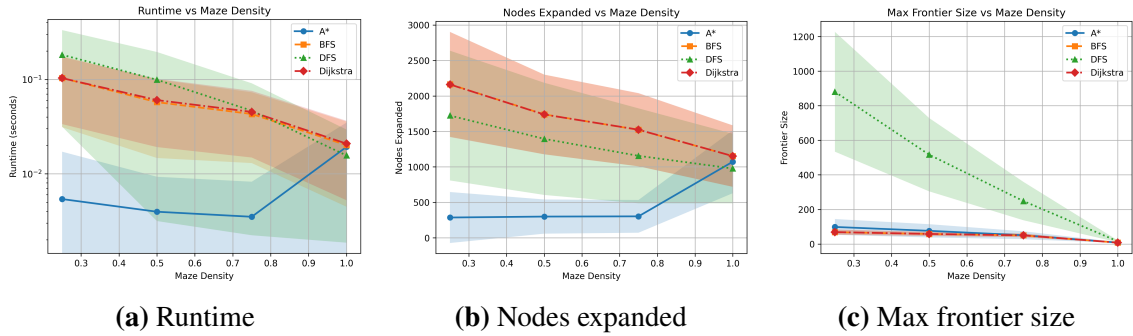


Figure 5.2: Scalability with wall density.

Interpretation. Classical planning is more expensive than a single greedy step, but A* keeps that cost manageable even on 81×81 grids. The cost is predictable and paid once per query. These results establish the baseline against which learning must compete.

5.4 Learning Stability and Convergence

5.4.1 Static weighted mazes

Table 5.2 reports convergence statistics for Q-learning on weighted static mazes. At size 11, all 100 runs converged within a mean of 117 episodes. At size 21, the convergence rate dropped to 85%. At size 31, only 61% converged, requiring a mean of 344 episodes when successful.

Table 5.2: Static Q-learning convergence (100 runs per size, 2000 max episodes).

Size	Converged	Mean episode	Median episode	Convergence (%)
11	100	117.0	100.0	100
21	85	214.1	150.0	85
31	61	344.3	300.0	61

The trend is clear: larger state spaces demand more exploration and yield lower reliability under a fixed training budget.

5.4.2 Dynamic mazes

Dynamic Q-learning behaves differently. All 100 runs converged at every tested size, but the required episode count increased with maze size: mean 255 at size 11, 445 at size 21, and 681 at size 31. The richer state representation and non-stationary dynamics slow learning, yet the training signal from bonuses, collisions, and goal rewards appears to stabilize the process.

Table 5.3: Dynamic Q-learning convergence (100 runs per size).

Size	Converged	Mean episode	Median episode	Convergence (%)
11	100	254.5	250.0	100
21	100	445.0	450.0	100
31	100	680.5	650.0	100

Interpretation. Learning incurs a substantial upfront sample cost even when it ultimately succeeds. On static maps, that cost is hard to justify when A* completes in milliseconds. In dynamic settings, the comparison shifts because the learned policy encodes reactions to moving obstacles.

5.5 Optimality Gap in Static Environments

Figure 5.3 and Table 5.4 summarize the path cost and length ratios relative to A*.

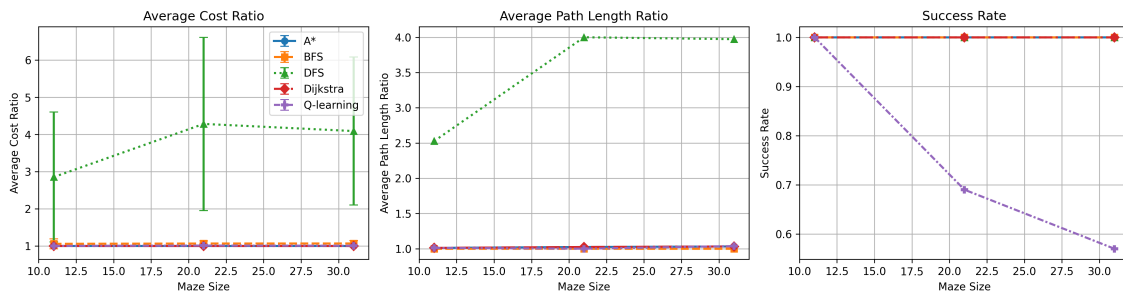


Figure 5.3: Path cost ratio (top) and path length ratio (bottom) versus maze size. Ratios are normalized by A* optimal cost on each instance.

Table 5.4: Mean path ratios and Q-learning success rate on weighted mazes (20 instances per size).

Size	Algorithm	Cost ratio	Length ratio	Success rate
11	A* / Dijkstra	1.00	1.01	1.00
11	BFS	1.06	1.00	1.00
11	DFS	2.85	2.53	1.00
11	Q-learning	1.00	1.01	1.00
21	A* / Dijkstra	1.00	1.02	1.00
21	BFS	1.06	1.00	1.00
21	DFS	4.28	4.00	1.00
21	Q-learning	1.01	1.00	0.69
31	A* / Dijkstra	1.00	1.03	1.00
31	BFS	1.07	1.00	1.00
31	DFS	4.09	3.97	1.00
31	Q-learning	1.00	1.04	0.57

A* and Dijkstra achieve unit cost ratio by construction. BFS incurs roughly 6% excess cost because hop-minimal paths diverge from cost-minimal paths when cell weights vary. DFS cost ratios exceed $2.8\times$ optimality even on small mazes and worsen beyond $4\times$ at sizes 21 and 31. Q-learning achieves near-optimal cost among successful runs, but its success rate declines as maze size grows.

Interpretation. When learning succeeds, it can match optimal planning on path cost. When it fails, the agent is effectively suboptimal. For safety-critical or cost-critical static routing, planning remains the more dependable choice.

5.6 Dynamic Environments: Planning Versus Learning

Figure 5.4 and Table 5.5 compare Q-learning, A* replanning, and the reactive agent after training.

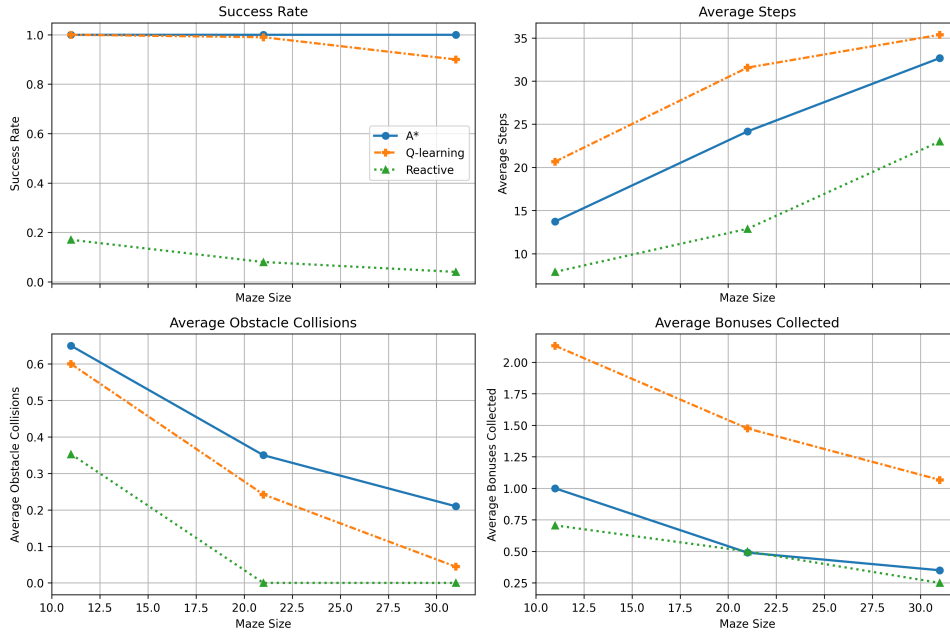


Figure 5.4: Dynamic environment metrics by maze size: success rate, mean steps, obstacle collisions, and bonuses collected.

Table 5.5: Dynamic comparison summary (100 runs per algorithm and size).

Size	Algorithm	Success	Steps	Collisions	Bonuses
11	A*	1.00	13.7	0.65	1.00
11	Q	1.00	20.7	0.60	2.13
11	R	0.17	7.9	0.35	0.71
21	A*	1.00	24.2	0.35	0.49
21	Q	0.99	31.6	0.24	1.47
21	R	0.08	12.9	0.00	0.50
31	A*	1.00	32.7	0.21	0.35
31	Q	0.90	35.4	0.04	1.07
31	R	0.04	23.0	0.00	0.25

A* replanning achieves perfect success at all sizes with the fewest steps. Q-learning approaches that reliability at smaller sizes while collecting more bonuses, but its success rate drops at the largest size. The reactive agent performs poorly, showing that one-step greedy rules are insufficient for this benchmark.

Interpretation. Dynamics change the trade-off but do not automatically favor learning. Full A* replanning remains the most reliable method for reaching the goal. Learning becomes attractive when the task reward structure includes bonuses that replanning underweights, although it comes with longer trajectories and some loss of reliability at the largest size.

5.7 Synthesis: Answering the Research Question

Planning is better when the environment is static and fully known, when optimality and certainty are required, and when a single query remains cheaper than RL training. Learning becomes advantageous when reward structure encourages exploration, when repeated replanning is costly, and when local perception must be integrated over time. The experiments show that the two paradigms are complementary rather than interchangeable.

CHAPTER 6. CONCLUSIONS

This chapter summarizes the main findings of the report and outlines the practical interpretation of the experiments.

6.1 Summary of Findings

The results show that planning and learning occupy complementary regions of a performance landscape shaped by maze scale, environmental dynamics, and reward structure. A* is the most efficient classical planner and remains the most reliable option on static maps. Q-learning can match near-optimal path cost when it converges, but its success rate falls as maze size increases. In dynamic environments, A* replanning remains the most dependable method for reaching the goal, while Q-learning collects more bonuses at the cost of longer trajectories and somewhat lower reliability.

6.2 Limitations

The study uses tabular Q-learning without function approximation, which limits scalability to larger grids. The experiments also use a controlled simulation setting rather than real-world sensing, so the results should be interpreted as a benchmark rather than a complete robotics evaluation.

6.3 Future Work

Future work could include larger mazes, deeper reinforcement learning methods, and incremental replanning approaches such as D* Lite. A broader comparison of training budgets, state representations, and reward schemes would also clarify the boundary between planning and learning in more complex navigation tasks.

REFERENCE

- [1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [3] S. Koenig and M. Likhachev, “D* lite,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2002, pp. 476–483.
- [4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [5] C. J. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, no. 3, pp. 279–292, 1992.
- [6] D. Silver, A. Huang, C. J. Maddison, et al., “Mastering the game of go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.
- [7] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, “Planning and acting in partially observable stochastic domains,” *Artificial Intelligence*, vol. 101, no. 1-2, pp. 99–134, 1998.
- [8] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.
- [9] D. Fox, J. H. Ko, K. Konolige, B. Limketkai, D. Schulz, and B. Stewart, “The dynamic weighted a* reinforcement learning algorithm,” *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 121–126, 2006.